

Internship report title

Internship report

Max Royer

Supervised by Marco Attene and Marco Livesu

ENS of Lyon
September 2025

1 Introduction

This report is here to wrap up my internship at the *CNR IMATI* in Genova, Italia. It was a pleasure to share those three months with Marco, Marco and the team (where there is 3 others Marco)!

My internship domain was computational geometry, the domain both Marco Attene and Marco Livesu are working on. Recently Lorenzo Diazzi, Marco A. et al published an article [2] talking about constrained Delaunay tetrahedrization. So far they had produced the robust tetrahedrization pipeline, using robust geometric predicates developped by Marco Attene. But they still lacked of a refinement process for the tetrahedrized output that other known software like TetGen [4], or CGAL [5].

Constrained Delaunay tetrahedrization (CDT) have a wide range of applications, they are commonly used in physics-based simulations such as the finite element method, solutions of partial differential equations or raytracing. However, these simulations and applications require high-quality meshes, which raw CDTs often fail to provide. Therefore, a refinement process is essential to improve the overall mesh quality and ensure the robustness and accuracy of the simulations.

The objective of this internship was to design and validate a greedy algorithm for refining Constrained Delaunay Tetrahedrizations (CDTs). We majorly gathered inspiration from the article [3] that was already proposing a refinement process but in their case they were authorizing themselves to deform the input (*i.e.* move input vertices in a small radius sphere). However in our case our constraints are strict, which means that we cannot remove a input vertex or change the geometry of the faces. We are only allowed to add vertices, which may lay on the surface of the model only if they do not change the original surface geometry.

The plan of this report is to first present some preliminaries and the state of the art, then to present the refinement process and finally to compare it with other ones that already exists.

2 Preliminary

2.1 Piecewise Linear Complex

To modelize any 3D model, polyhedra are often not enough to describe well the model (for example if the model have holes is not connected). This is why it's important to introduce Piecewise Linear Complex which have been.

Definition 2.1 (Piecewise Linear Complex). A Piecewise Linear Complex (PLC) X is a finite set of vertices, segments and facets of $\mathbb{R}^3$. A facet f is a polygonal region that can have holes and any number of sides, and so is not necessarily convex. Then X must respect those two rules:

- X must be closed by intersection, it means that when intersecting two elements of X the intersection is either empty or is an element of X .
- Every facet of X must be planar.

Here is an example of PLC in figure 1.

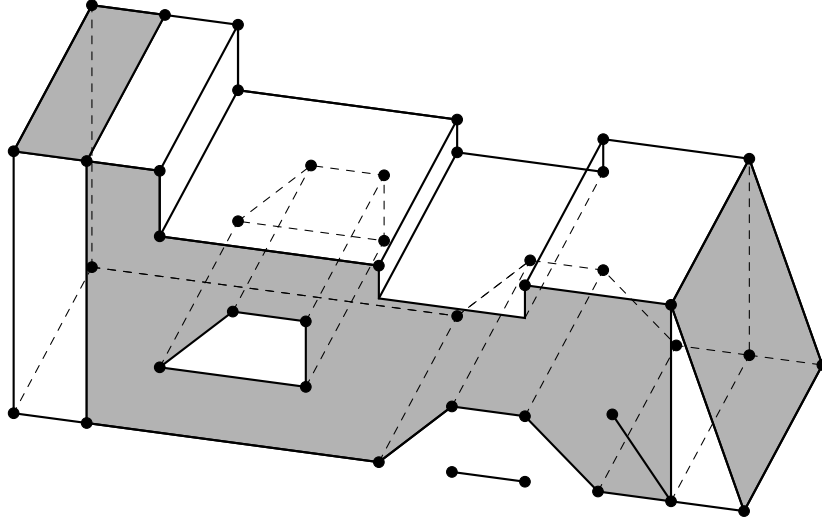


Figure 1: Example of PLC

2.2 Delaunay's triangulation

Definition 2.2 (Simplex). A simplex σ of dimension d is the convex hull of $(v_0, v_1, \dots, v_d)$ points of $\mathbb{R}^n$, such that $(v_1 - v_0, \dots, v_d - v_0)$ are linearly independant. The v_i 's are called the vertices of σ , and we say that σ is spanned by $V := \{v_0, \dots, v_d\}$. A face of σ is a simplex spanned by a subset of V , we denote the fact that τ is a face of σ by writing $\tau \leq \sigma$.

Remark 2.1. In dimension 1 a simplex is an segment, in dimension 2 it's a triangle and in dimension 3 it's a tetrahedra.

Definition 2.3 (Simplicial complex). Let K be a set of simplices, we say that K is a simplicial complex if:

- If $\sigma \in K$ then for all τ such that $\tau \leq \sigma$ we have $\tau \in K$.
- Let $\sigma, \sigma' \in K$ then $\sigma \cap \sigma'$ is either empty or a face of both.

Definition 2.4. Let K be a simplicial complex, and let $B \subseteq K$:

- The *closure* of B is :

$$\overline{B} := \{\tau \in K \mid \exists \sigma \in B, \tau \leq \sigma\}$$

- The *start* of B is :

$$\text{St}(B) := \{\sigma \in K \mid \exists \tau \in B, \tau \leq \sigma\}$$

- The *link* of B is :

$$\text{Lk}(B) := \overline{\text{St}(B)} \setminus \text{St}(\overline{B})$$

- The *vertices* of B are :

$$\text{Vert}(B) := \{v \in B \mid \dim v = 0\}$$

and the *edges* of B are :

$$\text{Edges}(B) := \{e \in B \mid \dim e = 1\}$$

Definition 2.5 (Delaunay triangulation). For $P \subseteq \mathbb{R}^d$, a valid Delaunay triangulation is a set $DT(P)$ of d -dimensional simplices such that for all $\sigma \in DT(P)$, there is no other points than the vertices that span σ that are in the *circum-hypersphere* of σ . The later condition is often denoted as the Delaunay condition.

Remark 2.2. In our case we'll only focus on dimension 3, in this case a Delaunay tetrahedrisation exists and is guaranteed to be unique if and only if the points are non coplanar and if there doesn't exist 5 points that are cospherical.

2.3 Constrained Delaunay tetrahedrisation

Definition 2.6 (Visible vertex). Let t be a tetrahedra, a vertex v is said to be visible from t if for any vertices of t one can draw a segment between it and v without intersecting t .

In the figure 2 v is visible whereas w is not because one the segment connecting to a vertex is intersecting the tetrahedra on the thick dotted part.

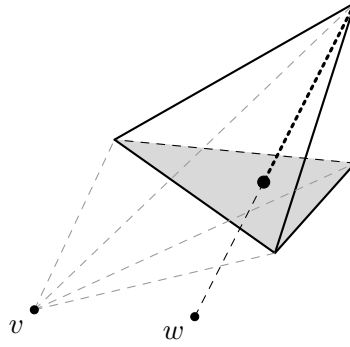


Figure 2: Example of visible vertex

In our specific case we are more interested by the Constrained Delaunay Triangulation (CDT) [1]. Indeed for example a polygon it's often that the Delaunay tetrahedrisation of his vertices does not includes all the sides of the polygon. This is why the constrained Delaunay Triangulation exists, it's to force the so called constrained Delaunay tetrahedrisations to contain some prescribed edges while maintaining *as much as possible* the Delaunay condition.

Definition 2.7 (Constrained Delaunay tetrahedrisation). Given a graph $G := (V, E)$ composed of vertices $V \subseteq \mathbb{R}^3$ and edges E , a valid constrained Delaunay tetrahedrisation $DT(G)$ is a set of tetrahedras such that:

- All vertices connected by an edge in G are also connected in at least one of the tetrahedras of $DT(G)$
- For all tetrahedras t of $DT(G)$ there is not another vertex v that is visible from t in the circum-sphere of t .

Remark 2.3. The obtained constrained Delaunay tetrahedrisation is a simplicial complex.

2.4 Quality Measure

To measure the quality of our refinement we need a quality measure. A good quality measure should be minimized on *well-shaped* tetrahedras and becomes really big on *bad-shaped* tetrahedras. Typically a *well-shaped* tetrahedra would be one close to be a regular tetrahedra, whereas the *bad-shaped* ones are typically slivers which are very flat tetrahedras that appears in constrained and unconstrained Delaunay tetrahedrisation (see on figure 3) or the ones with very acute angle.

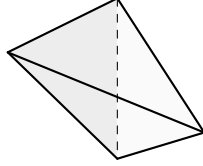


Figure 3: Example of sliver

In our implementation we choosed to use the Dirichlet energy because it's assigning 3 to a regular tetrahedra, is not upper bounded and assigns very high value to slivers.

Definition 2.8 (Edge matrix). Let t be a tetrahedra composed of vertices $(\mathbf{x}_i)_{0 \leq i \leq 3} \in \mathbb{R}^3$, the edge matrix of t is defined to be:

$$T := \begin{pmatrix} \mathbf{x}_1 - \mathbf{x}_0 & \mathbf{x}_2 - \mathbf{x}_0 & \mathbf{x}_3 - \mathbf{x}_0 \end{pmatrix}$$

Remark 2.4. This definition of the edge matrix is invariant by translation.

Definition 2.9 (Tetrahedra deformation). Let t be a tetrahedra and r another one that you want to deform t to, let T and respectively R be the associated edge matrix. We define the deformation matrix to be:

$$D := TR^{-1}$$

Definition 2.10 (Dirichlet energy). Let t be a tetrahedra and J be the deformation matrix when we deform t to the regular tetrahedra of side 1. We define the Dirichlet energy to be:

$$\mathcal{E}(t) := \frac{\text{Tr}(JJ^t)}{\det(J)^{\frac{2}{3}}}$$

Remark 2.5. The definition of this energy makes it so that it's invariant by rotation of t and of the regular tetrahedra chosen, this is why we say *the* regular tetrahedra of side 1.

Remark 2.6. One may wonder if the order of the vertices in the input tetrahedra t is important, the answer is that as soon as the tetrahedra is well-oriented the order does not change the result (hopefully). If the tetrahedra is badly-oriented the determinant at the denominator will be negative and so we just return the maximum value because it's out of definition for $\frac{2}{3}$ exponent.

3 State of the art

Right now the state of the art is mainly composed of two things. First there is TetGen who is doing a refinement process at the end of their tetrahedrisation. The thing is that TetGen fails on certain model of the Thingi10k 3D model set that we'll take as reference for the rest of the report. It's a free dataset composed of more than 4.000 various 3D models.

4 The algorithm

The algorithm is composed of several local operations that I will detail in the following. The main function is a full pass, which means it makes all the different local operations pass in a row. Then one can choose to only run this optimization pass a fixed number of times or to let it run until convergence (until no operations can be applied on the mesh).

The process for each individual operation pass is quite the same for all of them. First we compute the effect that the operation would have on each element of the mesh. This pre-computation in a sort simulate the effect that the local operation would have on this element and then return firstly if the operation is feasible (it could not be the case if the operation breaks the topology for example) and also if the operation is decreasing the quality measure in the local footprint of the operation.

This pre-computation yields us a queue of feasible operation, which we'll process in the following manner. Pop an element that is benefic to apply the operation on, apply the transformation, compute the other elements impacted by the operation and recompute the effect the same operation would have on them and add or remove those element of the queue accordingly.

The difficult part are often to know if the operation will break the topology, and to know which element are impacted by a local operation.

4.1 Edge split

The edge split as the name suggests is the fact to add a new vertex to an edge and then arrange the geometry accordingly. For this operation we do not have to worry about the fact that it could break the topology because it cannot. Then when splitting an edge (u, v) by adding a vertex w in the middle, the impacted edges are:

$$E := \text{Edges}(\overline{\text{St}(\{(u, w)\})}) \cup \text{Edges}(\overline{\text{St}(\{(w, v)\})})$$

Which are the union of all the edges of all the tetrahedras incident to either (u, w) or (w, v) . Here is an example of an edge split in the figure 4.

4.2 Edge collapse

The edge collapse may be one of the hardest operation to approach. The fact to collapse an edge (u, v) on u for example is just take all the edges incident to v and to change v in u . In this way the vertex v is removed along with all the tetrahedras incident to the edge (u, v) .

The difficulty of this operation is to prevent topological break. Firstly the vertex or the edge that we are going to remove should not be on the boundary of the model. Then the principal condition to ensure to not break the topology is the link condition for collapsing (u, v) :

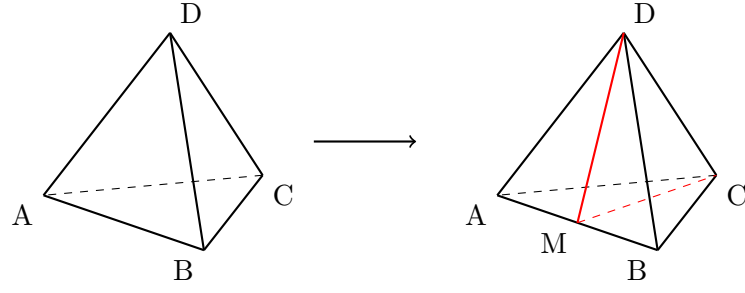


Figure 4: Example of splitting an edge

$$\text{Lk}(\{(u, v)\}) = \text{Lk}(\{u\}) \cap \text{Lk}(\{v\})$$

If this condition is not respected then the collapsing can't be made. In a second time when collapsing the edge (u, v) on u the removed edges are:

$$R := \text{Edges}(\text{St}(\{v\}))$$

and the edges to update are:

$$E := \text{TODO}$$

Here is an example of collapse in figure 5.

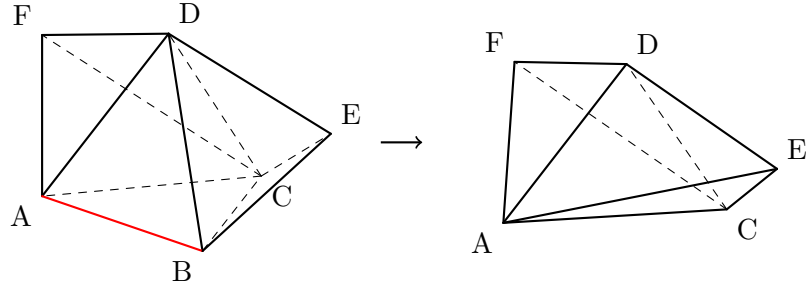


Figure 5: Example of collapsing an edge

4.3 Edge swap

4.4 Face swap

4.5 Vertex smoothing

4.6 Tetrahedra collapse

5 Result

6 Conclusion

References

- [1] L. P. Chew. “Constrained Delaunay triangulations”. In: *Proceedings of the Third Annual Symposium on Computational Geometry*. SCG ’87. Waterloo, Ontario, Canada: Association for Computing Machinery, 1987, pp. 215–222. ISBN: 0897912314. DOI: 10.1145/41958.41981. URL: <https://doi.org/10.1145/41958.41981>.
- [2] Lorenzo Diazzi et al. *Constrained Delaunay Tetrahedrization: A Robust and Practical Approach*. 2023. arXiv: 2309.09805 [cs.CG]. URL: <https://arxiv.org/abs/2309.09805>.
- [3] Yixin Hu et al. “Tetrahedral meshing in the wild”. In: *ACM Trans. Graph.* 37.4 (July 2018). ISSN: 0730-0301. DOI: 10.1145/3197517.3201353. URL: <https://doi.org/10.1145/3197517.3201353>.
- [4] Hang Si. “TetGen, a Delaunay-Based Quality Tetrahedral Mesh Generator”. In: *ACM Trans. Math. Softw.* 41.2 (Feb. 2015). ISSN: 0098-3500. DOI: 10.1145/2629697. URL: <https://doi.org/10.1145/2629697>.
- [5] The CGAL Project. *CGAL User and Reference Manual*. 6.0.1. CGAL Editorial Board, 2024. URL: <https://doc.cgal.org/6.0.1/Manual/packages.html>.